Apache Hadoop MapReduce

Bandwidth Utilisation Analysis of Web Server Logs using Python mrjob

Detail	Information
Name	Divya K N
USN	4GW23CI012
Assignment	Hadoop MapReduce: per-client bandwidth analysis
Platform	Apache Hadoop 3.4.3 (single-node, pseudo-distributed), HDFS, YARN
Language / library	Python 3.10 with mrjob 0.7.4
Operating system	Ubuntu Linux
Date	25 September 2026

1. Introduction

Apache Hadoop is an open-source framework for storing and processing very large datasets on a cluster of ordinary machines. Instead of pulling all the data onto one powerful computer, Hadoop divides files into blocks, spreads those blocks over many nodes and sends the program to the nodes that already hold the data. The same design scales from a single server to thousands of machines and keeps running when individual machines fail.

1.1 Hadoop Modules

Module	Responsibility
HDFS	Distributed storage layer. Files are cut into large blocks that are replicated across DataNodes, while the NameNode keeps the file-system metadata.
YARN	Resource management layer. The ResourceManager schedules applications and each NodeManager runs the tasks in containers.
MapReduce	Parallel processing model. The map phase turns input records into key-value pairs and the reduce phase aggregates the values of each key.
Hadoop Common	Shared libraries and utilities required by the other modules.

1.2 Daemons Running in This Setup

A single-node installation runs every daemon on the same machine. The five daemons below were listed by jps during this assignment (Figure 1), together with the process IDs that were observed.

Daemon	Function	Process ID
NameNode	Manages the HDFS namespace and knows where each block is stored	7111
DataNode	Stores the actual data blocks	7315
SecondaryNameNode	Periodically checkpoints the NameNode metadata	7554
ResourceManager	Allocates cluster resources to applications	7818
NodeManager	Launches and monitors containers on the node	8181

2. Objective

A web server records every request it receives in an access log. The aim of this assignment is to read such a log from HDFS and calculate how much bandwidth each client IP address has consumed. The calculation is expressed as a MapReduce program written with the Python mrjob library: the mapper extracts the client IP and the response size in bytes from each line, and the reducer adds up the bytes belonging to each IP address.

3. Environment Setup

3.1 Installation Layout

Hadoop 3.4.3 was installed in single-node (pseudo-distributed) mode under /home/aiml/hadoop. Four configuration files in the etc/hadoop directory control how the services behave.

Configuration file	Purpose
core-site.xml	Defines the default file system (fs.defaultFS)
hdfs-site.xml	HDFS options such as the replication factor and storage directories
mapred-site.xml	Selects YARN as the framework that runs MapReduce jobs
yarn-site.xml	YARN options such as the shuffle auxiliary service

3.2 Passwordless SSH

The Hadoop start scripts open SSH connections to launch each daemon, so an RSA key pair was generated and the public key was added to the list of authorised keys. This lets the scripts run without asking for a password.

```
sudo systemctl enable ssh
ssh-keygen -t rsa -P ""
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

3.3 Starting and Verifying the Services

HDFS and YARN were started with the scripts below, and jps was used to list the Java processes.

```
start-dfs.sh
start-yarn.sh
jps
```

The start scripts reported that the NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager were already running as processes 7111, 7315, 7554, 7818 and 8181. The jps output confirmed the same five daemons, so the cluster was ready for use (Figure 1). Later, the YARN web interface on port 8088 was used to track the submitted job (Figure 3).

4. Methodology

The program follows the standard three-stage MapReduce flow.

- **Map:** each log line is split on whitespace. The first token is the client IP address and the last token is the response size, so the mapper emits the pair (IP, bytes).
- **Shuffle and sort:** Hadoop collects every value that has the same IP key and hands the grouped values to a single reducer call.
- **Reduce:** the reducer sums the grouped values and emits (IP, total bytes).

The table below traces the 6-line HDFS log (Section 5) through these stages.

Client IP	Mapper output (bytes)	After shuffle	Reducer output
192.168.1.10	1200, 4300	[1200, 4300]	5500
192.168.1.20	2500, 7000	[2500, 7000]	9500
192.168.1.30	10000, 7500	[10000, 7500]	17500

5. Input Data and HDFS Loading

The log used for the Hadoop run contains six requests from three clients. It was stored in HDFS at /user/aiml/web_logs/access.log.

```
192.168.1.10 - - [09/Sep/2026:10:00:01 +0000] "GET /index.html HTTP/1.1" 200 1200
192.168.1.20 - - [09/Sep/2026:10:01:15 +0000] "GET /home.html HTTP/1.1" 200 2500
192.168.1.10 - - [09/Sep/2026:10:02:20 +0000] "GET /image.jpg HTTP/1.1" 200 4300
192.168.1.30 - - [09/Sep/2026:10:03:10 +0000] "GET /video.mp4 HTTP/1.1" 200 10000
192.168.1.20 - - [09/Sep/2026:10:04:25 +0000] "GET /style.css HTTP/1.1" 200 7000
192.168.1.30 - - [09/Sep/2026:10:05:30 +0000] "GET /script.js HTTP/1.1" 200 7500
```

The file was placed in HDFS with the following commands (Figure 2). The put command answered "File exists", which means that the copy already stored in HDFS was kept, and the cat command then displayed that stored copy.

```
hdfs dfs -mkdir -p /user/$USER/web_logs
hdfs dfs -put access.log /user/$USER/web_logs/
hdfs dfs -cat /user/$USER/web_logs/access.log
```

6. Implementation

The complete program is saved as bandwidth_analysis.py.

```
from mrjob.job import MRJob


class BandwidthAnalysis(MRJob):

    def mapper(self, _, line):
        try:
            parts = line.split()
            ip = parts[0]
            bytes_transferred = int(parts[-1])
            yield ip, bytes_transferred
        except (IndexError, ValueError):
            pass

    def reducer(self, ip, values):
        yield ip, sum(values)


if __name__ == '__main__':
    BandwidthAnalysis.run()
```

- **Defensive mapper:** a try/except block catches IndexError and ValueError, so blank or malformed lines are skipped instead of stopping the whole job.
- **Reducer:** sum(values) adds the byte counts that Hadoop has grouped under one IP address.
- **Entry point:** BandwidthAnalysis.run() lets the same file run with the inline runner for testing and with the hadoop runner for the cluster.

7. Execution

7.1 Local Test with the Inline Runner

A Python virtual environment was created and mrjob 0.7.4 was installed. The job was first tested locally, where mrjob processes the file on the same machine without using Hadoop (Figure 2).

```
python3 -m venv venv
source venv/bin/activate
pip install mrjob
python3 bandwidth_analysis.py access.log
```

This test read the local access.log, which holds eight requests dated 23 September 2026 (Figure 5). Its totals therefore differ from the HDFS run, and they are checked against their own input in Section 8.

```
192.168.1.10 - - [23/Sep/2026:10:15:32 +0530] "GET /index.html HTTP/1.1" 200 1024
192.168.1.20 - - [23/Sep/2026:10:16:10 +0530] "GET /image.jpg HTTP/1.1" 200 2048
192.168.1.10 - - [23/Sep/2026:10:17:05 +0530] "GET /about.html HTTP/1.1" 200 512
192.168.1.20 - - [23/Sep/2026:10:18:22 +0530] "GET /video.mp4 HTTP/1.1" 200 4096
192.168.1.30 - - [23/Sep/2026:10:19:15 +0530] "GET /home.html HTTP/1.1" 200 3072
192.168.1.10 - - [23/Sep/2026:10:20:01 +0530] "GET /style.css HTTP/1.1" 200 256
192.168.1.30 - - [23/Sep/2026:10:21:45 +0530] "GET /script.js HTTP/1.1" 200 1024
192.168.1.20 - - [23/Sep/2026:10:22:12 +0530] "GET /logo.png HTTP/1.1" 200 512
```

7.2 Running on Hadoop

The job was then submitted to the cluster with the hadoop runner, reading the log directly from HDFS.

```
python3 bandwidth_analysis.py -r hadoop hdfs:///user/$USER/web_logs/access.log
```

Figure 3 shows how the run progressed:

- mrjob detected Hadoop 3.4.3 at /home/aiml/hadoop/bin/hadoop and located hadoop-streaming-3.4.3.jar.
- The job connected to the ResourceManager at 0.0.0.0:8032 and was submitted as job_1790308889194_0002.
- One input file was found and divided into two splits.
- Progress moved from map 0% reduce 0% to map 100% reduce 100%, and the job completed successfully.

The main counters reported at the end of the run (Figure 4) are summarised below.

Counter	Value	Meaning
Map input records	6	One record per log line
Map output records	6	One (IP, bytes) pair per line
Map output bytes	121	Size of the mapper output
Merged map outputs / shuffled maps	2	Two map outputs fetched by the reducer
Reduce input groups	3	Three distinct client IPs
Reduce output records	3	One total per client IP
Failed shuffles / shuffle errors	0	No errors during the shuffle
CPU time spent (ms)	1760	Total CPU time of the job

After the counters, mrjob streamed the final output from HDFS and removed its temporary directories. The totals were written to output.txt, which is shown in Figure 6.

8. Results and Verification

8.1 Hadoop Run (HDFS log, 6 requests)

Client IP	Requests	Calculation (bytes)	Total bandwidth
192.168.1.10	2	1200 + 4300	5500
192.168.1.20	2	2500 + 7000	9500
192.168.1.30	2	10000 + 7500	17500
Total	6		32500

8.2 Local Run (local log, 8 requests)

Client IP	Requests	Calculation (bytes)	Total bandwidth
192.168.1.10	3	1024 + 512 + 256	1792
192.168.1.20	3	2048 + 4096 + 512	6656
192.168.1.30	2	3072 + 1024	4096
Total	8		12544

8.3 Observations

- In both runs, the output of the job matches a manual calculation from its own input file, and the grand total equals the sum of all response sizes in that log.
- The counters of the Hadoop run agree with the data: six map input records for six log lines, and three reduce groups for three client IPs.
- In the HDFS log, 192.168.1.30 used the most bandwidth (17500 bytes) because it requested the large video.mp4 file.

9. Conclusion

A single-node Hadoop 3.4.3 cluster was set up, HDFS and YARN were started and verified, and a Python mrjob program was written to calculate the bandwidth used by each client IP address. The program was tested locally and then executed on the cluster, where it read its input from HDFS and completed successfully. The results matched manual calculations and the job counters, which confirms that the mapper, the shuffle and the reducer work correctly. The screenshots supporting each step are given in the annexure that follows.

Annexure: Execution Screenshots



Figure 1: Passwordless SSH setup, starting HDFS and YARN, and verifying the daemons with jps

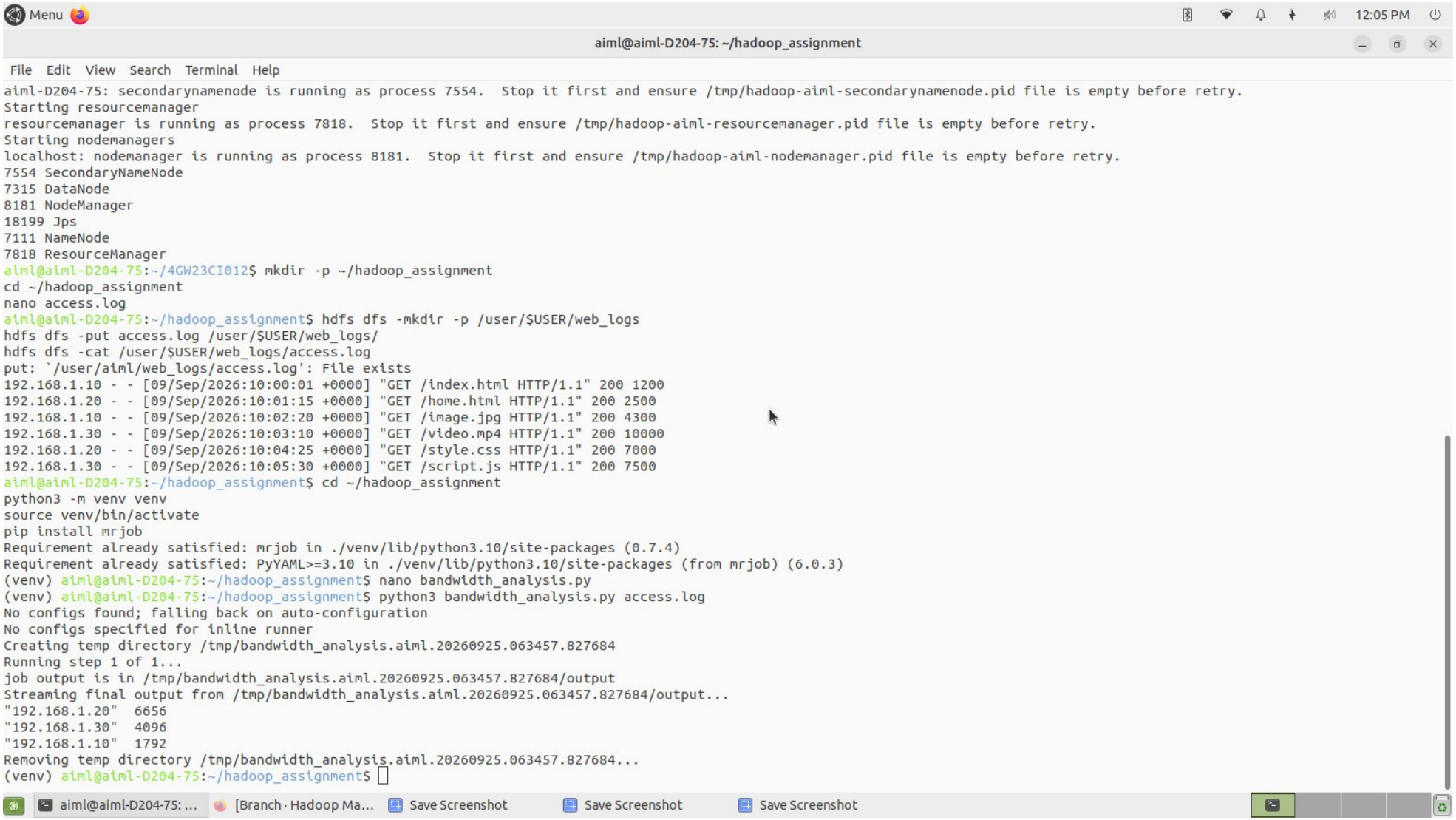


Figure 2: Loading access.log into HDFS, creating the virtual environment and running the job locally

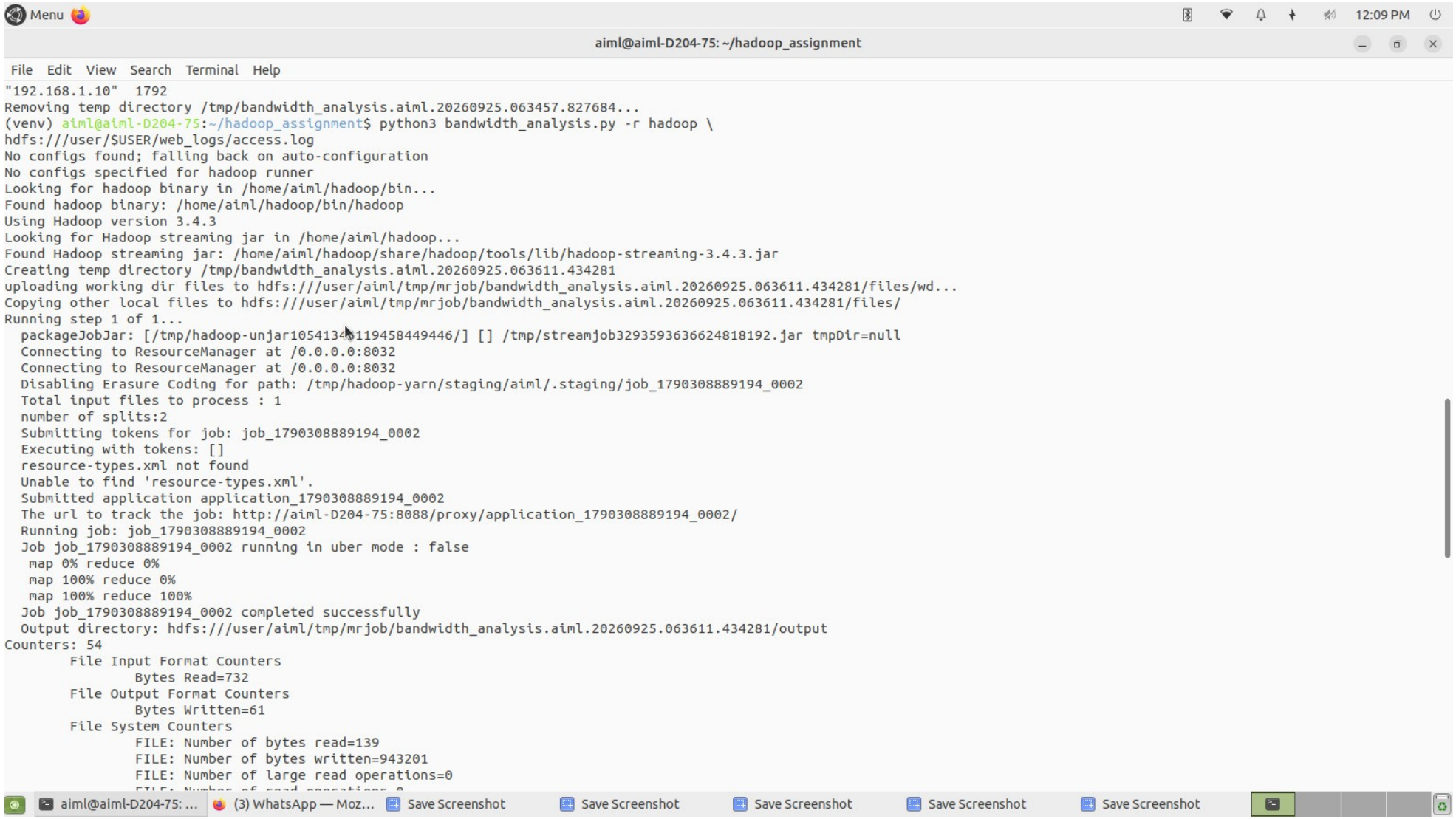


Figure 3: Hadoop streaming job submitted to YARN (map 100%, reduce 100%, job completed successfully)



Figure 4: Job counters and the final output streamed back from HDFS

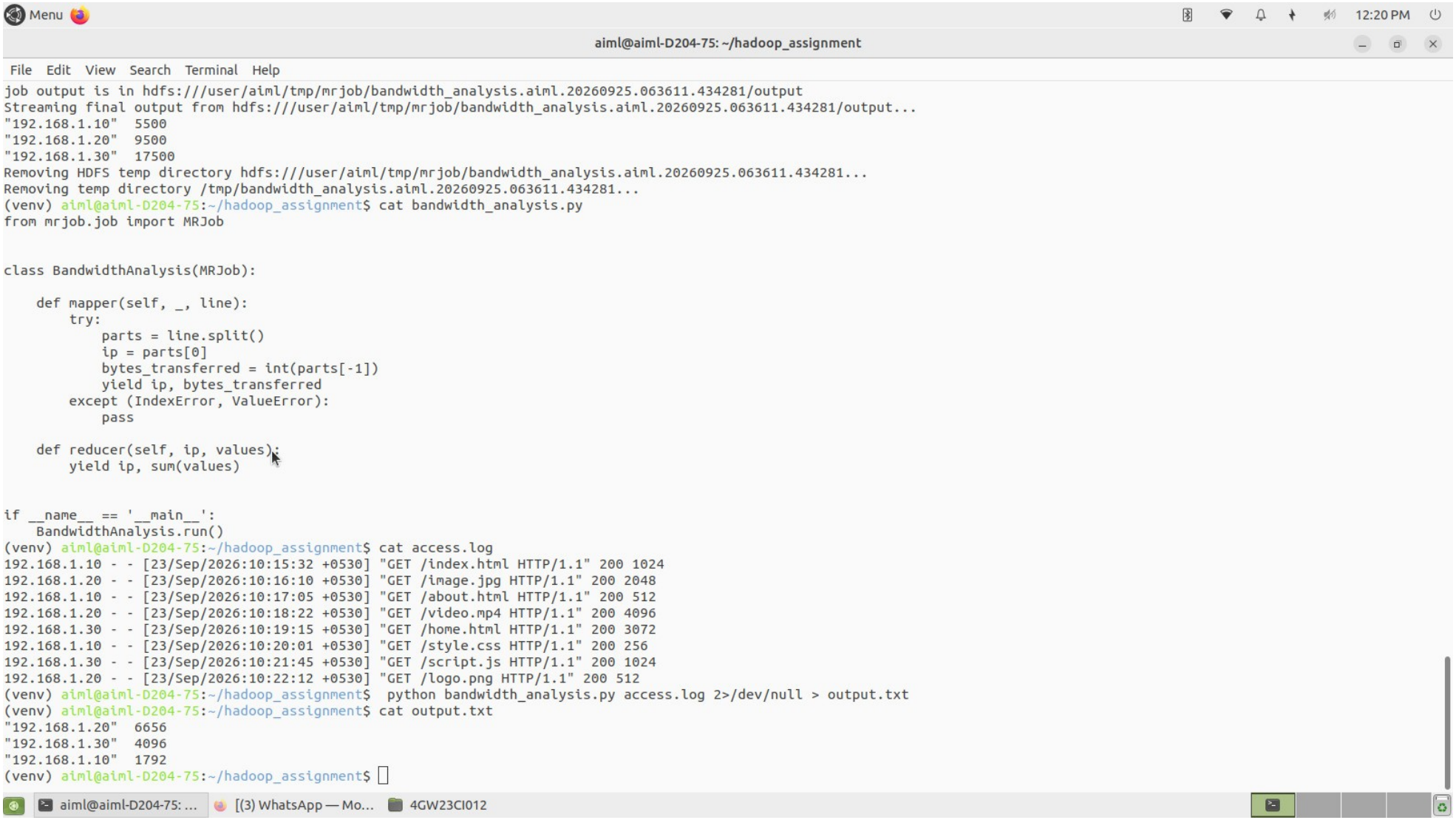


Figure 5: Source code, local test log and local-run output in the terminal

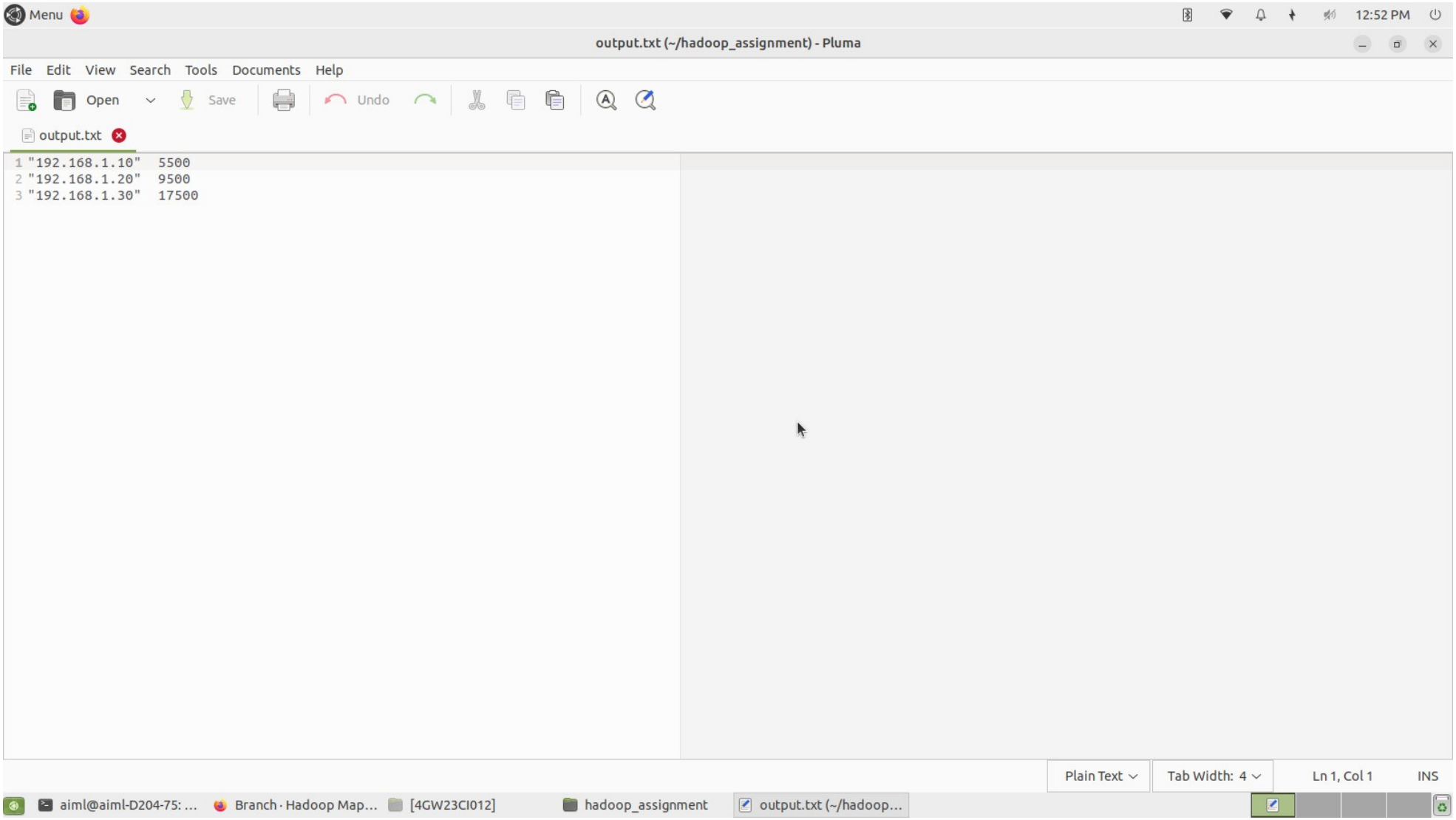


Figure 6: Contents of output.txt containing the bandwidth totals